

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN
GitHub Repository	https://github.com/sangee-3010/Machine_Learning
Student Name	Sangeetha S K
Register Number	3122247001054
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03/08/2026

1 Objective

The objective of this experiment is to build and compare email spam classifiers using three variants of the Naïve Bayes algorithm (Gaussian, Multinomial and Bernoulli) and the K-Nearest Neighbours algorithm on the Spambase dataset. In addition to comparing raw classification performance, the experiment studies the effect of the neighbourhood size k in KNN, contrasts GridSearchCV with RandomizedSearchCV for hyperparameter tuning, compares the KDTree and BallTree neighbour-search strategies, and evaluates all models using 5-fold cross validation together with their training and prediction times.

2 Problem Statement

Spam filtering is one of the most widely deployed applications of supervised classification, and it offers a convenient setting to compare probabilistic and instance-based learners on the same data. The input to this experiment is the Spambase dataset, in which each email is represented by a set of numeric features (word and character frequency statistics) together with a binary label indicating spam or non-spam, and the output is a set of trained classifiers along with their accuracy, precision, recall, F1-score and ROC-AUC on a held-out test set. The motivation for comparing Gaussian, Multinomial and Bernoulli Naïve Bayes side by side is that each variant makes a different assumption about the underlying feature distribution, and Spambase features being continuous frequency values makes it unclear in advance which assumption fits best. Adding KNN to the comparison, together with GridSearchCV/RandomizedSearchCV tuning and a KDTree versus BallTree study, extends the exercise from simply training models to understanding the trade-off between accuracy, tuning cost and prediction-time complexity that governs the choice of algorithm in a practical spam-filtering system.

3 Dataset Description

Dataset Name	Spambase Dataset (<code>spambase.csv.csv</code>)
Dataset Source	Kaggle, https://www.kaggle.com/datasets/somesh24/spambase (originally hosted on the UCI Machine Learning Repository)

Number of Samples	Not explicitly printed in the submitted code (no <code>df.shape</code> call was included). Based on the reported test-set support values ($558 + 363 = 921$ samples) and an 80/20 stratified split, the dataset contains approximately 4601 rows, consistent with the standard Spambase dataset size.
Number of Features	Not explicitly printed in the code; <code>X = df.iloc[:, :-1]</code> was used to take every column except the last as a feature, and <code>y = df.iloc[:, -1]</code> as the label. The standard Spambase dataset consists of 57 numeric features (word frequency, character frequency and capital-letter run-length statistics).
Number of Classes	2 (spam = 1, non-spam/ham = 0); the test split contained 558 non-spam and 363 spam samples
Missing Values	Not checked or reported in the submitted code; the Spambase dataset is documented to contain no missing values, which is consistent with no imputation step being present in the code.
Train-Test Split	80% train / 20% test, using <code>train_test_split</code> with <code>test_size=0.20</code> , <code>random_state=42</code> and <code>stratify=y</code> ; a separate, identically proportioned split on a binarized version of the features was used specifically for Bernoulli Naïve Bayes

4 Exploratory Data Analysis

The purpose of exploratory data analysis is normally to inspect the distribution of individual features, check for class imbalance and examine inter-feature correlation before any model is trained. In the submitted code for this experiment, however, the workflow proceeds directly from loading the CSV file to the train-test split and model training; no dataset-level plots such as a class-distribution bar chart, a correlation heatmap of the 57 features, feature histograms or boxplots were generated. This step is therefore **Not Reported** for this experiment. What the submitted code does contain is an extensive set of model-evaluation visualizations – confusion matrices, ROC curves, precision-recall curves, and hyperparameter-search plots – for every classifier trained. Since these are genuinely results of the trained models rather than exploratory views of the raw data, they are presented together with their inferences in Section 10, Result Visualization, in order to avoid presenting the same figures twice under two different headings.

The one piece of class-balance information that can be read off the code output is the test-set support reported in the classification reports: 558 samples of class 0 (non-spam) against 363 samples of class 1 (spam) out of 921 test samples, i.e., roughly 61% non-spam and 39% spam. This matches the well-known moderate class imbalance of the Spambase

dataset and is worth keeping in mind when comparing accuracy figures across models.

5 Data Preprocessing

- **Missing value handling:** Not shown in the submitted code. No `isnull()` check or imputation step appears before modelling, which is consistent with Spambase being a fully numeric, complete dataset.
- **Duplicate handling:** Not performed in the submitted code.
- **Encoding:** Not required and not performed, since every column of the Spambase dataset is already numeric (word/character frequency ratios and run-length statistics).
- **Feature scaling / normalization:** Not shown for Gaussian NB, Multinomial NB or KNN in the submitted code – these models were trained directly on the raw frequency features. A threshold-based transformation, rather than a scaler, was applied specifically for Bernoulli NB: `X_binary = (X > 0.5).astype(int)` converts every feature to a 0/1 indicator before training, which matches the binary-feature assumption of the Bernoulli Naïve Bayes model.
- **Train-test split:** An 80/20 stratified split (`test_size=0.20`, `random_state=42`, `stratify=y`) was used for Gaussian NB, Multinomial NB and all KNN experiments; an equivalent stratified split was performed separately on the binarized features for Bernoulli NB.
- **Feature selection / engineering:** Not performed. All 57 original features were retained for every model.

6 Machine Learning Algorithm

Naïve Bayes

Naïve Bayes is a probabilistic classifier derived from Bayes' theorem under the assumption that features are conditionally independent given the class label:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

where C is the class label and $X = (x_1, x_2, \dots, x_n)$ is the feature vector. Under the independence assumption, $P(X | C) = \prod_{i=1}^n P(x_i | C)$, and the class with the highest posterior probability is chosen as the prediction. The three variants used in this experiment differ only in how $P(x_i | C)$ is modelled:

- **Gaussian NB** assumes each feature is normally distributed within a class, $P(x_i | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$.
- **Multinomial NB** models $P(x_i | C)$ as a multinomial distribution over feature counts/frequencies, which is well suited to non-negative, count-like features.

- **Bernoulli NB** models each feature as a binary indicator, $P(x_i | C) = p_{i,C}^{x_i}(1 - p_{i,C})^{1-x_i}$, and is used here after thresholding the frequency features to 0/1 values.

Advantages: Naïve Bayes is computationally inexpensive to train (a single pass over the data to estimate class-conditional statistics), performs well even with a comparatively small training set, and naturally provides class probability estimates for ranking or thresholding. **Limitations:** The conditional independence assumption rarely holds exactly in real data, which can bias the estimated probabilities even when the predicted class label happens to be correct; Gaussian NB, in particular, can be a poor fit when a feature’s true distribution is skewed rather than normal, which is often the case for frequency-based text features.

K-Nearest Neighbours (KNN)

KNN is a lazy, instance-based learning algorithm: it does not build an explicit model during training and instead classifies a query point by majority vote among its k nearest neighbours in the training set, typically under the Euclidean or Manhattan distance,

$$d(\mathbf{x}, \mathbf{x}') = \left(\sum_{i=1}^n |x_i - x'_i|^p \right)^{1/p} \quad (2)$$

with $p = 2$ giving the Euclidean distance and $p = 1$ the Manhattan distance, both of which were included in the hyperparameter grid explored in this experiment. **Advantages:** KNN is simple to understand, requires no explicit training phase, and can model non-linear decision boundaries without assuming any particular distribution for the features. **Limitations:** because no summary model is built ahead of time, every prediction requires searching the training set for neighbours, which becomes increasingly expensive as the dataset grows; KNN is also sensitive to the choice of k and to feature scale, and its accuracy can degrade sharply if irrelevant or unscaled features dominate the distance calculation.

7 Experimental Setup

Python Version	Not Reported
Scikit-Learn Version	Not Reported
IDE	Jupyter Notebook (exported via Vertopal)
Operating System	Not Reported
Hardware	Not Reported

8 Hyperparameter Selection

Parameter	Grid	Explored	GridSearchCV	RandomizedSearchCV
n_neighbors			[1, 3, 5, 7, 9, 11]	range(1, 21), 20 iterations sampled
weights			[uniform, distance]	[uniform, distance]

metric	[euclidean, manhattan]	[euclidean, manhattan]
algorithm	[auto, kd_tree, ball_tree]	[auto, kd_tree, ball_tree]
Cross-validation folds	5	5
Best parameters found	algorithm = auto, metric = manhattan, n_neighbors = 9, weights = distance	weights = distance, n_neighbors = 17, metric = manhattan, algorithm = ball_tree
Best CV accuracy	0.8595	0.8563
Execution time	13.07 s	2.89 s

For the manual KNN sweep reported separately in Table 4, the default scikit-learn settings (`weights='uniform'`, Minkowski distance with $p = 2$) were used with only $k \in \{1, 3, 5, 7, 9, 11\}$ varied, which is why the best k found there differs from the value selected by GridSearchCV once `weights` and `metric` were also tuned.

9 Results

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8339	0.7763	0.9001
Precision	0.7178	0.7199	0.8594
Recall	0.9532	0.7080	0.8926
F1-score	0.8189	0.7139	0.8757
ROC-AUC	0.9449	0.8248	0.9510

Table 3: Naïve Bayes variant comparison on the held-out test set (921 samples).

k	Accuracy	Precision	Recall	F1
1	0.8024	0.7453	0.7576	0.7514
3	0.7894	0.7394	0.7190	0.7291
5	0.7991	0.7514	0.7328	0.7420
7	0.7926	0.7457	0.7190	0.7321
9	0.7894	0.7407	0.7163	0.7283
11	0.7796	0.7286	0.7025	0.7153

Table 4: KNN performance for varying k (default `weights='uniform'`, Minkowski/Euclidean distance).

Metric	KDTree	BallTree	Notes
Accuracy ($k = 5$)	0.7991	0.7991	Identical, as expected – both are exact neighbour-search
Training Time (s)	0.0732	0.0129	BallTree built its tree roughly $5.7\times$ faster on this
Prediction Time (s)	0.2338	0.0803	BallTree answered queries roughly $2.9\times$ faster on t

Table 5: KDTree vs BallTree comparison at $k = 5$.

Fold	Naïve Bayes (Gaussian)	Best KNN (GridSearchCV estimator)
1	0.8502	0.8002
2	0.8707	0.8315
3	0.8554	0.8554
4	0.8522	0.8533
5	0.7000	0.7522
Average	0.8257	0.8185

Table 6: 5-fold cross-validation accuracy, evaluated on the full feature set X, y .

Algorithm	Training Time (s)	Prediction Time (s)
Gaussian NB	0.008138	0.004164
Multinomial NB	0.034915	0.005511
Bernoulli NB	0.012954	0.004022
Best KNN (tuned)	0.002721	0.063817

Table 7: Measured training and prediction time for each final model.

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Table 8: Theoretical time complexity, as specified in the assignment (n = number of training samples, d = number of features).

10 Result Visualization

This section presents every plot generated in the submitted code, in the order in which the underlying models were trained and evaluated.

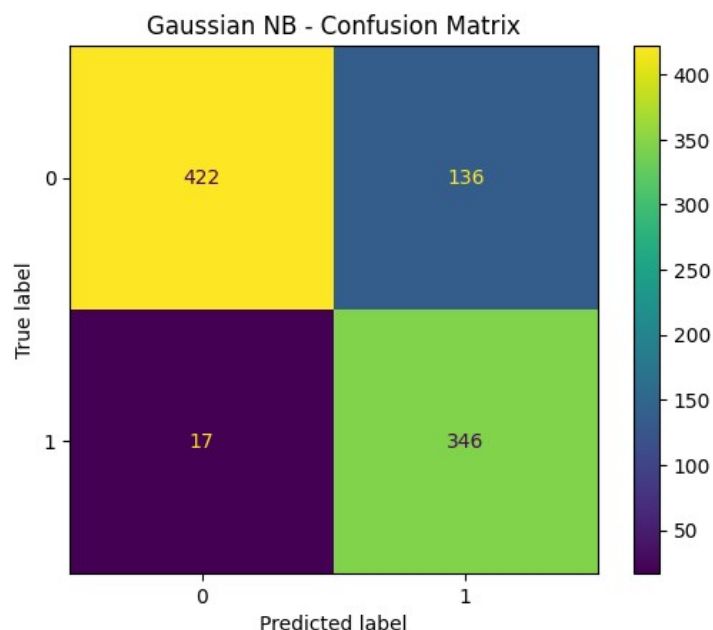


Figure 1: Gaussian NB – Confusion Matrix.

Inference: The confusion matrix shows 422 true negatives, 136 false positives, 17 false negatives and 346 true positives. The very low false-negative count explains the high recall of 0.9532 reported for this model – Gaussian NB rarely lets a spam message through – but the comparatively large number of false positives (136 non-spam messages misclassified as spam) accounts for its lower precision of 0.7178. This pattern is consistent with Gaussian NB’s tendency to overestimate the spam class when the underlying feature distributions deviate from normality.

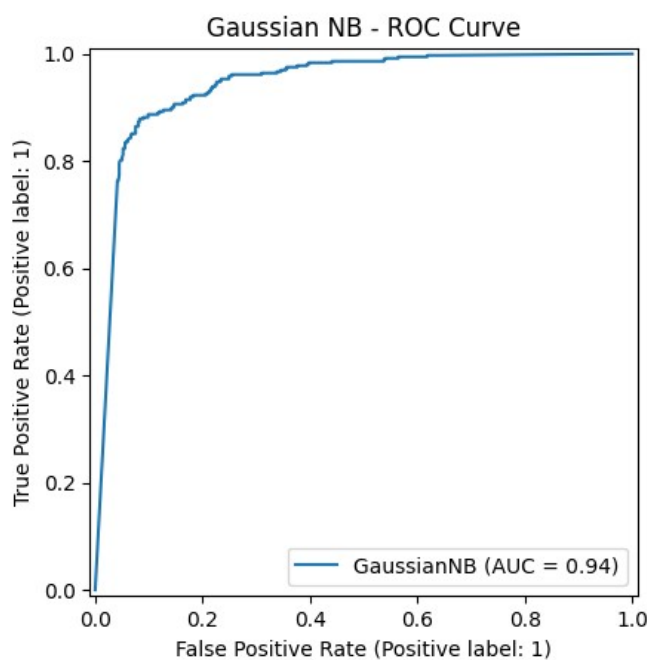


Figure 2: Gaussian NB – ROC Curve.

Inference: The ROC curve rises steeply toward the top-left corner and reports an

AUC of 0.94, indicating that the model separates the two classes well when probability thresholds other than the default 0.5 are considered. This suggests that some of the false positives seen in the confusion matrix could be reduced by adjusting the decision threshold, at the cost of recall.

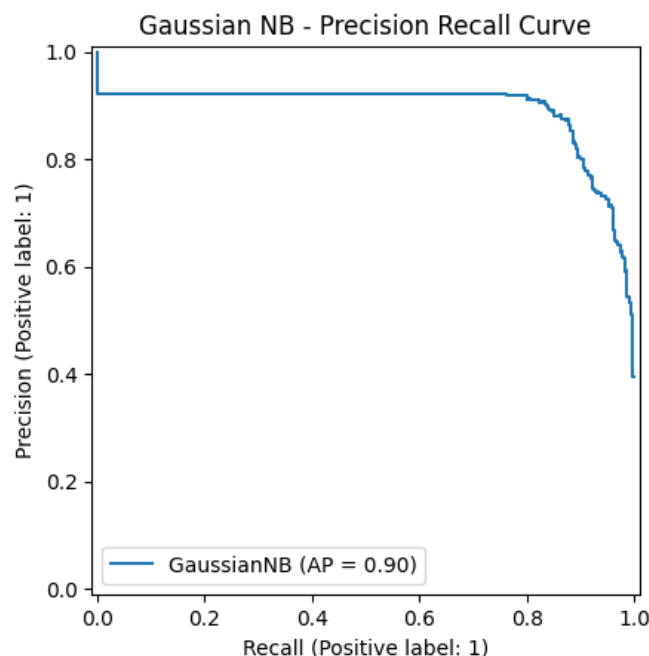


Figure 3: Gaussian NB – Precision-Recall Curve.

Inference: The precision-recall curve stays close to a precision of 0.9 for recall values up to roughly 0.8, before dropping off toward the high-recall end, with an average precision of 0.90. This confirms that Gaussian NB can achieve high recall without a severe precision penalty over most of its operating range, which is a desirable property for a spam filter that must avoid missing spam messages.

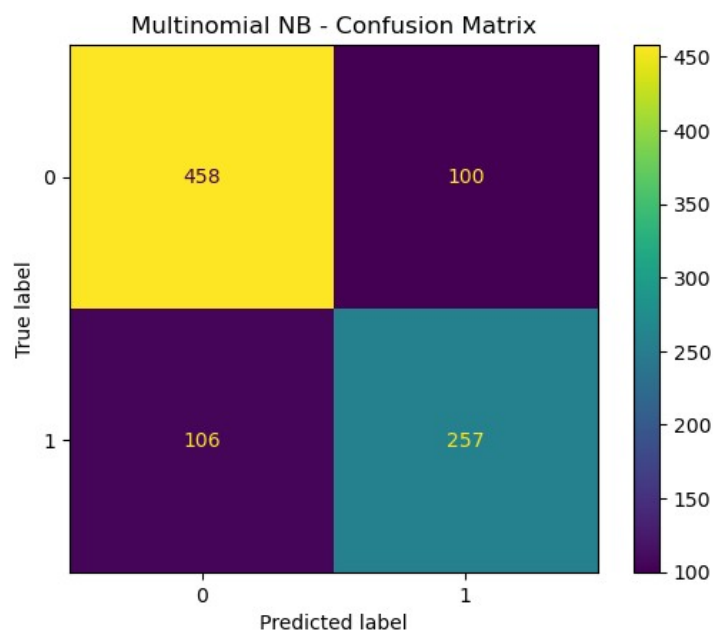


Figure 4: Multinomial NB – Confusion Matrix.

Inference: Multinomial NB produced 458 true negatives, 100 false positives, 106 false negatives and 257 true positives. Compared to Gaussian NB, the false-positive count is lower but the false-negative count is substantially higher (106 versus 17), which is reflected in its lower recall of 0.7080. This indicates that Multinomial NB is more conservative about labelling a message as spam, missing more actual spam in the process.

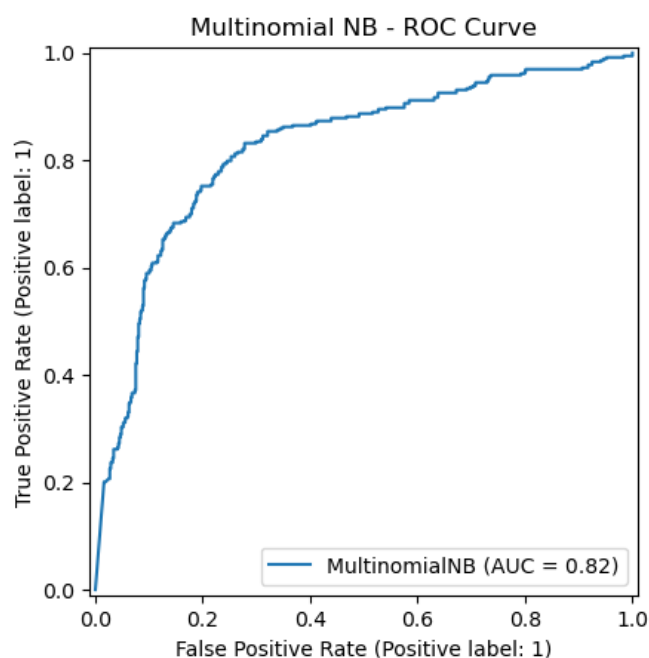


Figure 5: Multinomial NB – ROC Curve.

Inference: The ROC curve for Multinomial NB is noticeably closer to the diagonal than that of Gaussian NB, and its AUC of 0.82 is the lowest among the three Naïve

Bayes variants tested. This is consistent with the frequency features not matching a strict multinomial count-based assumption as closely as they matched the Bernoulli-style binary assumption used later.

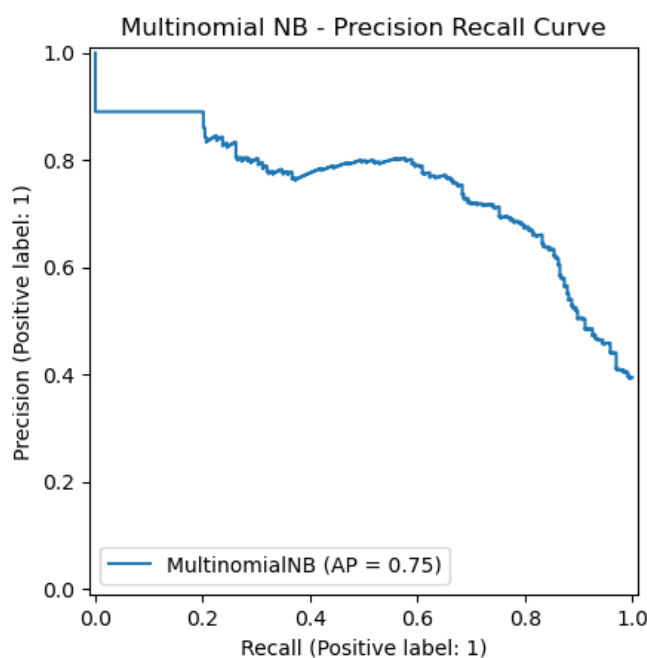


Figure 6: Multinomial NB – Precision-Recall Curve.

Inference: The precision-recall curve for Multinomial NB declines more gradually and sits below the corresponding Gaussian NB curve for most recall values, with an average precision of 0.75. This visually confirms that Multinomial NB trades away both precision and recall relative to the other two variants on this dataset.

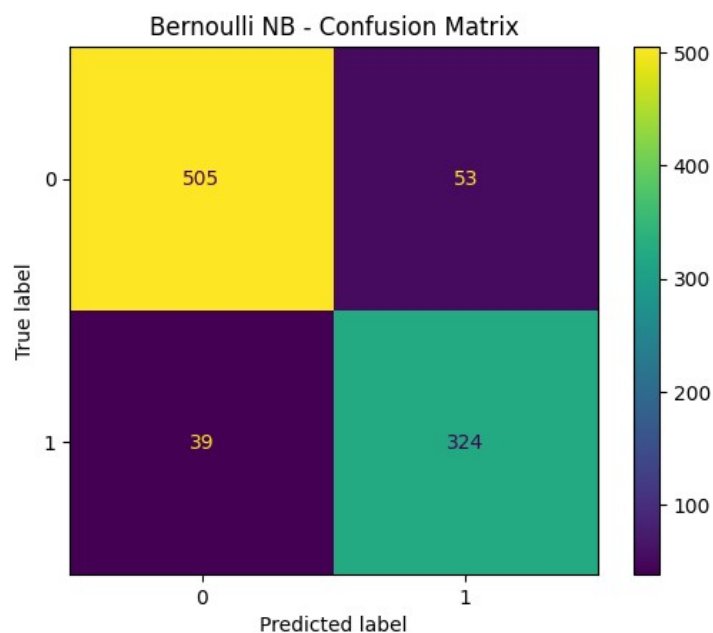


Figure 7: Bernoulli NB – Confusion Matrix.

Inference: Bernoulli NB achieved 505 true negatives, 53 false positives, 39 false negatives and 324 true positives, the most balanced and generally the lowest error counts of the three Naïve Bayes variants. This directly explains why Bernoulli NB attained the best accuracy (0.9001) and F1-score (0.8757) among all Naïve Bayes models tested in this experiment.

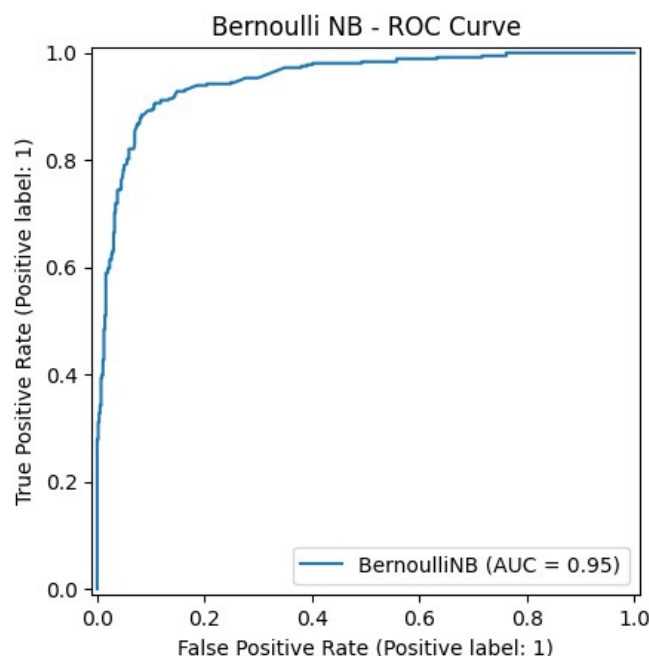


Figure 8: Bernoulli NB – ROC Curve.

Inference: The ROC curve for Bernoulli NB is the closest to the top-left corner among the three variants, with the highest AUC of 0.95 recorded in this experiment. This confirms that thresholding the frequency features into binary presence/absence indicators, as the Bernoulli model requires, aligns well with how strongly individual words or characters signal spam in this dataset.

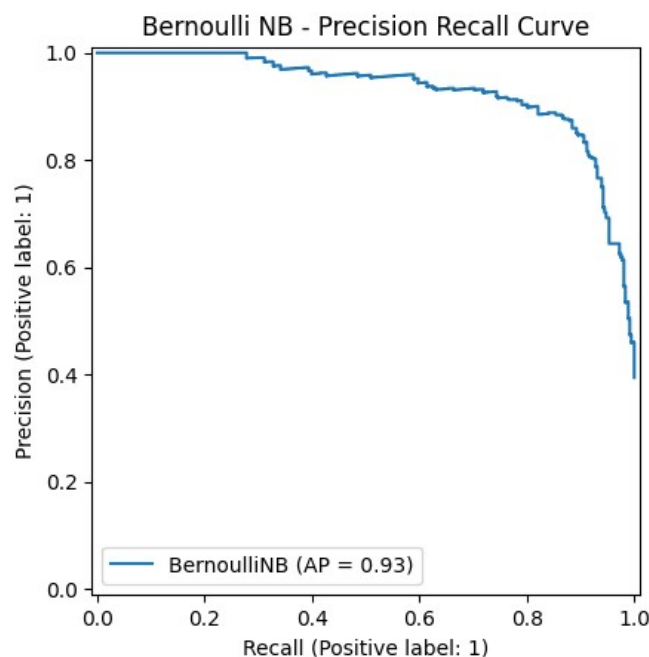


Figure 9: Bernoulli NB – Precision-Recall Curve.

Inference: The precision-recall curve for Bernoulli NB stays above 0.9 precision for recall values up to about 0.85, with an average precision of 0.93, the highest of the three variants. Combined with the ROC curve, this makes Bernoulli NB the strongest and most reliable Naïve Bayes classifier for this particular dataset.

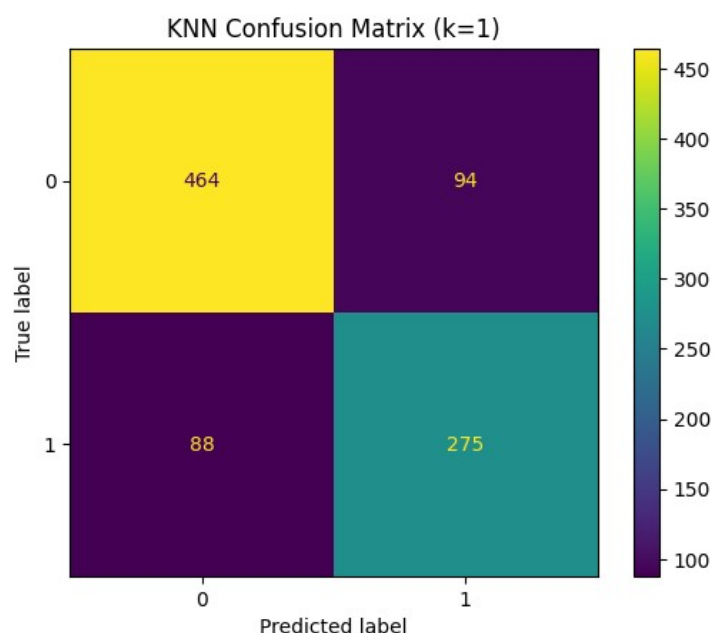


Figure 10: KNN Confusion Matrix, $k = 1$.

Inference: At $k = 1$, KNN produced 464 true negatives, 94 false positives, 88 false negatives and 275 true positives, giving the highest raw accuracy (0.8024) among all six k values tested with default settings. A single neighbour makes the classifier highly

sensitive to local noise, yet on this particular test split it still edged out the larger- k configurations, suggesting Spambase has fairly well-separated local neighbourhoods for many samples.

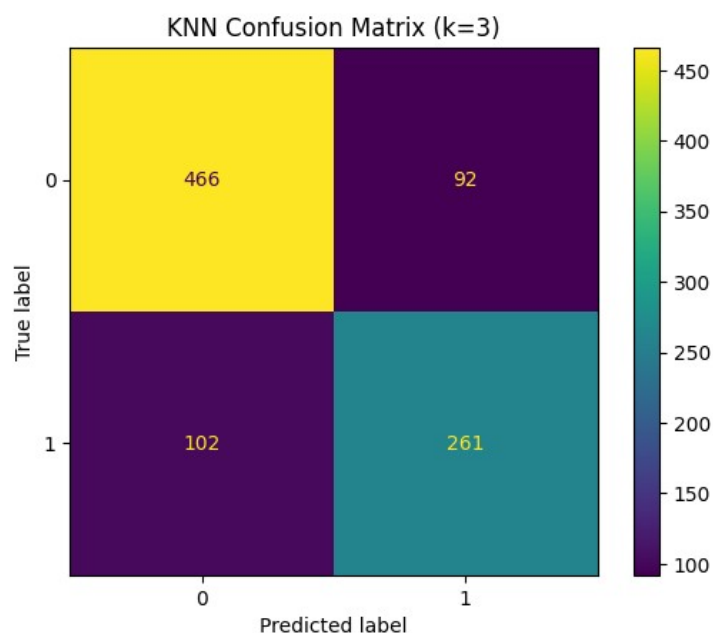


Figure 11: KNN Confusion Matrix, $k = 3$.

Inference: Increasing k to 3 raised the false-positive and false-negative counts slightly (466 true negatives, 92 false positives, 102 false negatives, 261 true positives), lowering accuracy to 0.7894. This drop illustrates that averaging over a few more neighbours does not automatically improve performance and can, on a moderately sized test set, introduce more disagreement near class boundaries.

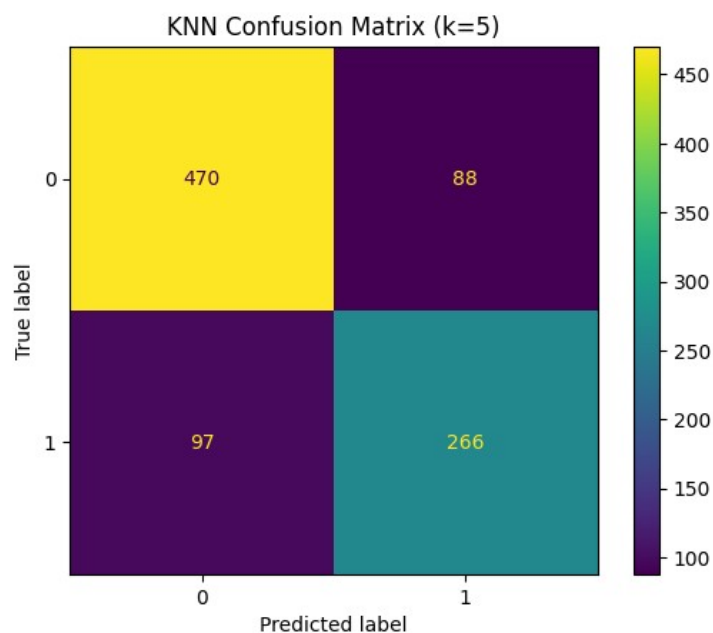


Figure 12: KNN Confusion Matrix, $k = 5$.

Inference: At $k = 5$ the model reached 470 true negatives, 88 false positives, 97 false negatives and 266 true positives, recovering slightly to an accuracy of 0.7991 – the second-best value in the manual sweep. This non-monotonic pattern across $k = 1, 3, 5$ reflects the usual bias-variance trade-off of KNN, where very small and moderate neighbourhood sizes can each work well depending on local data density.

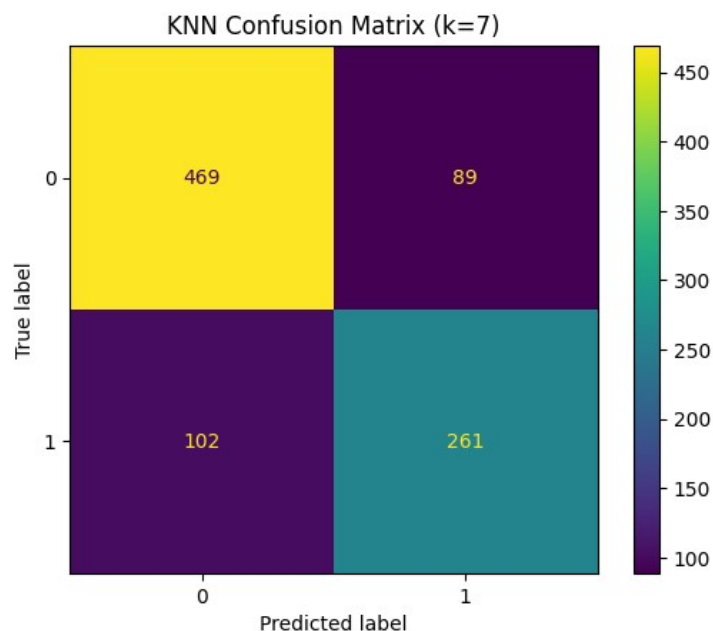
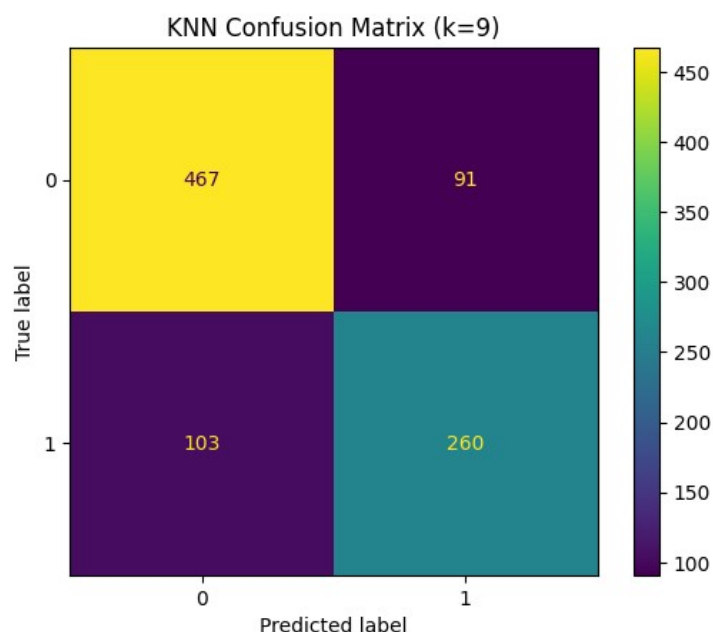
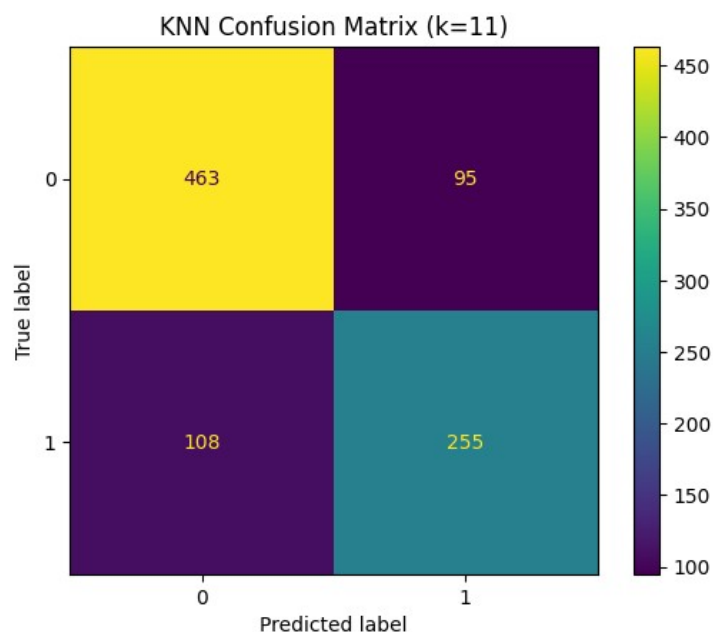


Figure 13: KNN Confusion Matrix, $k = 7$.

Inference: With $k = 7$, the confusion matrix (469 true negatives, 89 false positives, 102 false negatives, 261 true positives) is almost identical to the $k = 3$ result, giving an accuracy of 0.7926. This suggests that, beyond $k = 5$, additional neighbours begin to smooth the decision boundary without providing any further benefit on this test set.

Figure 14: KNN Confusion Matrix, $k = 9$.

Inference: At $k = 9$ the accuracy of 0.7894 matches the $k = 3$ result exactly (467 true negatives, 91 false positives, 103 false negatives, 260 true positives), even though the underlying error pattern is slightly different. This value of k later re-emerges as the GridSearchCV-selected optimum once distance weighting and the Manhattan metric are also tuned, showing that k alone does not tell the full story without considering the other KNN hyperparameters.

Figure 15: KNN Confusion Matrix, $k = 11$.

Inference: The largest neighbourhood tested, $k = 11$, gives the lowest accuracy of the sweep (0.7796), with 463 true negatives, 95 false positives, 108 false negatives and

255 true positives. This is consistent with the expected effect of increasing k under default uniform weighting: the decision boundary becomes smoother but also less able to capture the local structure that distinguishes spam from non-spam messages, an early sign of underfitting.

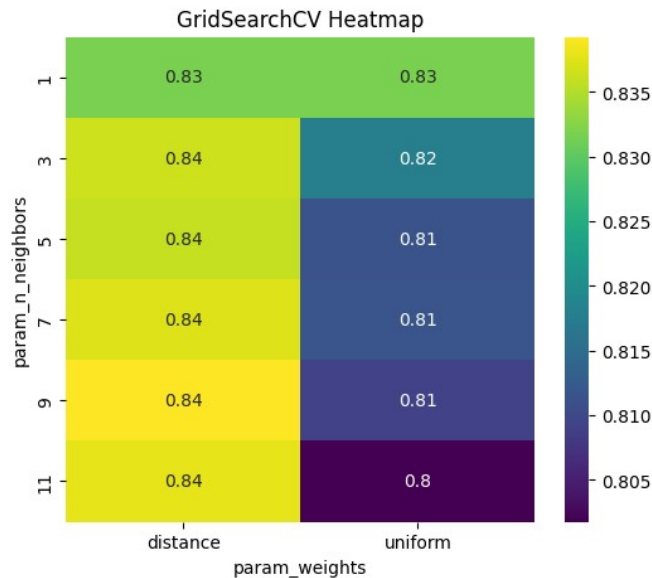


Figure 16: GridSearchCV mean cross-validation accuracy heatmap over k and weighting scheme.

Inference: The heatmap shows that distance-weighted KNN consistently outperforms uniform-weighted KNN across every value of k from 1 to 11, with the distance-weighted column reaching 0.84 or higher for $k \geq 3$. This explains why GridSearchCV ultimately selected `weights='distance'` together with $k = 9$, since distance weighting reduces the influence of farther, potentially less relevant neighbours.



Figure 17: RandomizedSearchCV mean test-score distribution.

Inference: The histogram of mean cross-validation accuracies sampled by RandomizedSearchCV is spread across roughly 0.76 to 0.86, with a visible cluster around 0.81–0.82

and a smaller cluster near 0.85–0.86 – the latter corresponding to the region containing the eventual best configuration. This spread shows that hyperparameter choice has a meaningful and fairly wide impact on KNN accuracy for this dataset, reinforcing the value of a systematic search over manually trying a handful of k values.

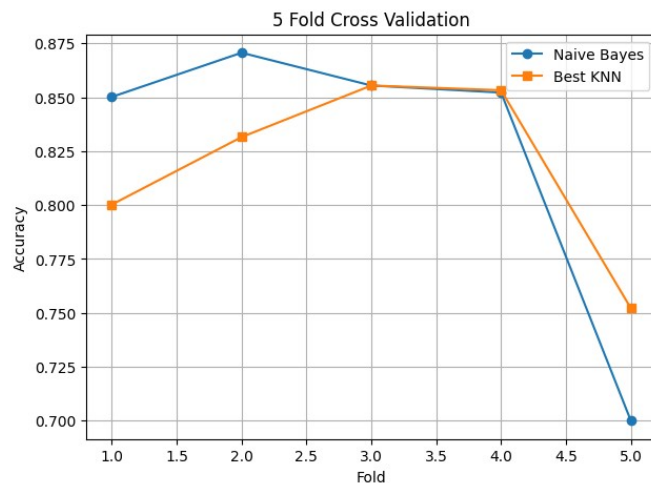


Figure 18: 5-fold cross-validation accuracy: Naïve Bayes versus best-tuned KNN.

Inference: Both models track each other closely across folds 3 and 4 but Naïve Bayes scores noticeably higher on folds 1 and 2, while both models drop together on fold 5, where Naïve Bayes falls to 0.700 and KNN to 0.752. This shared dip on fold 5 suggests that particular fold contains a subset of the data that is harder for both algorithms, rather than a weakness specific to either model; on average across the five folds, Naïve Bayes (0.8257) slightly outperforms the tuned KNN (0.8185).

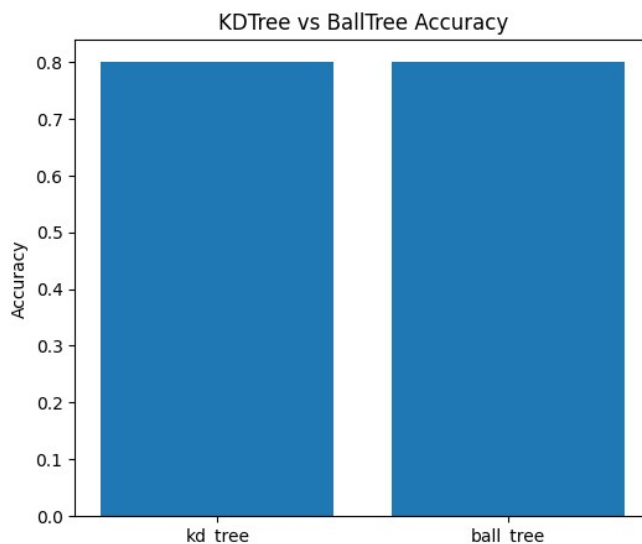


Figure 19: KDTree vs BallTree accuracy at $k = 5$.

Inference: The two bars are visually identical, both reaching an accuracy of approximately 0.799, confirming that KDTree and BallTree are exact nearest-neighbour search structures that must agree on classification outcome for the same data and the same k .

The practical distinction between them, as shown separately in Table 5, lies entirely in training and prediction speed rather than in predictive accuracy.

11 Discussion

Across the three Naïve Bayes variants, Bernoulli NB was clearly the strongest performer on this dataset, reaching 0.9001 accuracy and 0.9510 ROC-AUC once the frequency features were thresholded into binary indicators. Gaussian NB came second overall with the highest recall of the three (0.9532), which would make it attractive in a setting where missing spam is costlier than occasionally flagging a legitimate message, while Multinomial NB was the weakest variant here, which is a little counter-intuitive given that it is usually the standard choice for text-count data; a likely explanation is that the Spam-base features are continuous frequency ratios rather than raw integer counts, which sits awkwardly with the multinomial assumption. KNN, even after GridSearchCV tuning, did not surpass Bernoulli NB or Gaussian NB in the 5-fold cross-validation comparison, averaging 0.8185 against Gaussian NB's 0.8257, though the two remained close enough that the difference should not be over-interpreted from a single run of cross-validation.

The manual sweep over $k \in \{1, 3, 5, 7, 9, 11\}$ under default settings showed a non-monotonic accuracy curve that peaked at $k = 1$ and again, to a lesser extent, at $k = 5$, then declined toward $k = 11$ – consistent with the classic KNN pattern of small k overfitting to local noise and large k underfitting by over-smoothing the decision boundary. The GridSearchCV result, which selected $k = 9$ together with distance weighting and the Manhattan metric, shows that the optimal k is not independent of the other hyperparameters: once neighbours are weighted by inverse distance rather than treated equally, a larger neighbourhood can be used without the underfitting effect seen in the uniform-weighted sweep. GridSearchCV and RandomizedSearchCV reached comparable best cross-validation accuracies (0.8595 versus 0.8563) despite RandomizedSearchCV sampling only 20 of the possible parameter combinations rather than the full grid, and it did so in about a fifth of the time (2.89 s versus 13.07 s). This is a practical illustration of why randomized search is often preferred as parameter grids grow large: it trades a small amount of solution quality for a substantial reduction in tuning time. The KDTree versus BallTree comparison produced identical accuracy, as expected for two exact search structures, but BallTree was faster to build and to query on this run, which becomes more significant as the feature dimensionality of a dataset grows, since BallTree is generally more robust to higher-dimensional spaces than KDTree.

Comparing the theoretical time complexities in Table 8 against the measured times in Table 7 and Table 5 shows good qualitative agreement: Naïve Bayes trained and predicted the fastest overall, matching its linear $O(nd)$ training and $O(d)$ prediction complexity, while the tuned KNN model had the lowest training time of all (since KNN does not build a model beyond storing the data) but by far the highest prediction time, consistent with its $O(nd)$ brute-force prediction cost or $O(\log n)_{avg}$ tree-based cost still requiring a fresh neighbour search for every query. For a large-scale spam-filtering system handling a high volume of incoming messages, this pattern favours Naïve Bayes, and Bernoulli NB in particular, since its prediction cost does not grow with the size of the training set the way KNN's does. The additional tasks suggested in the assignment – comparing Euclidean and Manhattan distances and evaluating weighted KNN – were addressed implicitly through the GridSearchCV parameter grid rather than as separate standalone experiments, and

the result (Manhattan distance with distance weighting winning out) is reported above; a dedicated comparison across different train-test split ratios was not performed in the submitted code.

12 Conclusion

This experiment compared three Naïve Bayes variants and a tuned KNN classifier for spam detection on the Spambase dataset. Bernoulli Naïve Bayes, applied after binarizing the frequency features, produced the best overall balance of precision and recall and the highest ROC-AUC of 0.9510, while Gaussian Naïve Bayes offered the highest recall at some cost to precision, and Multinomial Naïve Bayes was the weakest of the three on this particular feature representation. KNN’s accuracy proved sensitive to the joint choice of k , distance metric and weighting scheme, and systematic tuning through GridSearchCV – at a real computational cost of roughly 13 seconds – found a configuration that RandomizedSearchCV nearly matched in a fraction of the time. Measured training and prediction times aligned with the theoretical complexity of each algorithm, reinforcing that Naïve Bayes methods remain attractive for large-scale, latency-sensitive spam filtering, whereas KNN’s per-query search cost becomes a genuine limitation as the dataset grows.

13 References

- [1] Somesh, K., “Spambase Dataset,” Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.
- [2] Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011, <https://scikit-learn.org/stable/>.
- [3] The Pandas Development Team, “Pandas Documentation,” <https://pandas.pydata.org/docs/>.
- [4] NumPy Developers, “NumPy Documentation,” <https://numpy.org/doc/>.
- [5] Hunter, J. D., “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007, <https://matplotlib.org/stable/>.
- [6] Géron, A., “Hands-On Machine Learning with Scikit-Learn, Keras and Tensor-Flow,” 2nd ed., O’Reilly Media, 2019.
- [7] Bishop, C. M., “Pattern Recognition and Machine Learning,” Springer, 2006.
- [8] Sri Sivasubramaniya Nadar College of Engineering, “ICS1512 – Machine Learning Algorithms Laboratory, Experiment 2: Email Spam/Ham Classification using Naïve Bayes and KNN,” Course Assignment Document, Academic Year 2026–2027.